EPSR-test-ex40 readxyz: cell read from standard input

Program(s): readxyz | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises the file-conversion program `readxyz`, which turns an `.xyz` coordinate file into an EPSR `.ato` configuration [1] in which every atom is a separate one-atom molecule, with a non-cubic box header and default Lennard-Jones parameters per element. Here the second line of the `.xyz` file holds no cell, so the six cell parameters are read from standard input, including a triclinic cell and the retry after a read error.

Code exercised

Fortran (original, fortran_src/):

- `readxyz.f`: `run_readxyz` (command line, file search), `readxyz` (parsing of the `.xyz` file, cell from line 2 or stdin, one molecule per atom)
- `auxiliary_routines.f`: `convertalphabetagamma` (a b c alpha beta gamma -> EPSR cell angles)
- `ato_manipulation_rev4.f`: `ctype`, `ftype` (atom types with default parameters), `centreofmass`
- `ato_read_and_save.f`: `moloutc` (writing the `.ato`, non-cubic header)

Rust (port, rust/epsr-core/src/):

- `tools/readxyz.rs`: `run`, `readxyz`, `set_type_parameters` (the `ctype` calls)
- `tools/groupc_common.rs`: `convertalphabetagamma`, `setup_lattice`, `centreofmass`, `moloutc`, `chsymbol`/`chatwt` (103 elements)
- `ato.rs`: `State::ctype`, `State::ftype`

Physical system

Single molecules from the EPSR-test DMSO/water example: one dimethyl sulphoxide molecule (10 atoms: S, 2 C, O, 6 H) and one water molecule (O, 2 H), as `.xyz` files exported from the `DMSO.ato` and `WaterMolecule.ato` templates. DMSO is placed in a triclinic cell a, b, c = 20, 22, 25 Å with alpha, beta, gamma = 80°, 95°, 100° (written by `convertalphabetagamma` as the EPSR cell angles 100°, 10.547°, 118.435°), water in a cubic 30 Å cell; temperature 300 K (default). Each element becomes one atom type with default parameters (O: eps 0.65 kJ/mol, sigma 3.1 Å; C: 0.8, 3.7; H: 0, 0 with mass 2).

Inputs

`DMSO.xyz` and `WaterMolecule.xyz` copied from EPSR-test. `readxyz DMSO.xyz < readxyz_dmso.stdin` (one line `20 22 25 80 95 100`); `readxyz WaterMolecule < readxyz_water.stdin` (no extension given; first line `bad values here` is not numeric and the cell is asked again, then `30.0 30.0` / `30.0 90 90 90` spread over two lines, list-directed read).

How the accuracy was verified

`commands.txt` (2 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. `readxyz`/`readpdb` crash (SIGSEGV) as compiled from the original source, so the reference build adds an explicit INTERFACE block for `ctype` (`rust/tools/build_patched_readxyz_readpdb.sh`); the port behaves like the patched program. Result of the final regression run of the complete port: every file identical in 1 pass.
Outputs compared: `DMSO.ato` and `WaterMolecule.ato` (10 and 3 one-atom molecules). Fortran reference: the original `readxyz`/`readpdb` binaries crash (SIGSEGV) at the first call of `ctype`, which has optional arguments but is called with 11 of its 13 arguments without an explicit interface (gfortran passes the hidden character lengths where `radmaxsph`/`pseudofacsph` are expected). The reference was therefore produced with an explicit `INTERFACE` block for `ctype` added to the program (no other change; absent arguments mean `radmaxsphere` = `0.5*sigcor`, `pseudofacmol` = `1`), built by `rust/tools/build_patched_readxyz_readpdb.sh`; the comparison is against this patched build, not the original binary.

Notes

Output is a starting configuration only; the user normally regroups atoms into molecules afterwards.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.